Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2233 — Programación Avanzada 2026-2

Publicación: 5 de noviembre de 2026
Actividad evaluada

Actividad 8
Interfaces Gráficas 2

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: `Actividades/AC8`
- **Fecha máxima de entrega:** 5 de noviembre a las 17:10 hrs
- **Ejecución de actividad**: La Actividad será ejecutada **únicamente** desde la terminal del computador. **Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.**

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (***push***), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Instrucciones

Tu Gato de Halloween fue todo un éxito, pero tus amigos ya quieren jugar otra cosa y, sobre todo, quieren hacerlo cada uno desde su computador. Como quedaron calabazas de sobra, decides programar **DCCalabazaPong**: un Pong en línea para dos jugadores, donde cada jugador mueve su paleta para que la calabaza no salga por su lado de la cancha.

Por suerte, ya programaste el servidor y las ventanas. El servidor se encarga de la sala, de la física de la pelota y de los goles, y **no debes modificarlo**. Tu trabajo es **conectar el cliente**: escuchar al servidor sin congelar la interfaz, reaccionar a sus mensajes mediante señales y usar `QTimer` para la paleta, el reloj y los avisos.



Figura 1: Ventana de Juego de DCCalabazaPong durante una partida

Archivos entregados

- `servidor/`: el servidor completo y `bot.py`, un rival de prueba. **No debes modificar esta carpeta ni su contenido.**
- `cliente/frontend.py`: las dos ventanas, ya construidas. **Debes completarlo.**
- `cliente/backend.py`: el *back-end*, con las señales de `Logica` ya declaradas. **Debes completarlo.**
- `cliente/main.py`: crea la aplicación, el *back-end* y las ventanas. **Debes completarlo.**
- `protocolo.py`, `parametros.py` y `assets/`, también en `cliente/`: las funciones `enviar` y `recibir`, las constantes del juego y la imagen de la pelota. **No debes modificarlos.**

Las partes por completar están marcadas con el comentario `# Completar`, y las del bonus con `# Bonus`. No debes crear archivos nuevos ni cambiar los nombres entregados.

Para probar tu programa, abre una terminal por programa, siempre en el orden indicado:

```
# Terminal 1, en la carpeta servidor/
python3 main.py

# Terminal 2, en la carpeta servidor/ (rival de prueba)
python3 bot.py

# Terminal 3, en la carpeta cliente/
python3 main.py
```

Consideraciones

- En vez del *bot*, también puedes abrir un segundo cliente en otra terminal.
- Entre pruebas, no reinicies el servidor ni el *bot*: el *bot* vuelve solo a la sala cuando cierras tu cliente.
- El código entregado ya se puede ejecutar: la Ventana de Inicio aparece, pero todavía no hace nada.
- Si prefieres dos clientes, primero detén el *bot*, porque si no el segundo recibirá *La sala está llena.*

Modelación del programa

La separación entre *back-end* y *front-end* ya viene hecha, y debes mantenerla: las ventanas no saben nada de *sockets* y no se comunican directamente entre ellas. Además, debes respetar la regla de oro de la semana:

Solo el *thread* principal toca la interfaz. Los demás *threads* le avisan mediante señales.

Importante

Ten en cuenta que la función `recibir` de `protocolo.py` es bloqueante: espera hasta que llegue un mensaje. Por eso, solo debes llamarla desde `EscucharQThread`. Si se pierde la conexión, levanta `ConnectionError` u `OSError`. Para enviar mensajes, usa el método `enviar_comando` de `Logica`, que ya está implementado y maneja la desconexión. Puedes llamarlo desde el *thread* principal.

Mensajes con el servidor

Todos los mensajes son diccionarios serializados con JSON y codificados en UTF-8, que se envían con las funciones de `protocolo.py`. Las posiciones corresponden a la esquina superior izquierda de cada elemento, relativas a la cancha. En las listas de dos elementos, el primero corresponde al jugador de la izquierda.

Mensaje	**Significado**
Del cliente al servidor	
`{"comando": "unirse", "nombre": str}`	Pide entrar a la sala.
`{"comando": "mover", "y": int}`	Informa la nueva posición de tu paleta.
Del servidor al cliente	
`{"tipo": "aceptado"}`	Entraste a la sala.
`{"tipo": "rechazado", "razon": str}`	No pudiste entrar a la sala.
`{"tipo": "inicio", "nombres": [str, str], "duracion": int}`	Empieza la partida. `duracion` está en segundos.
`{"tipo": "estado", "pelota": [x, y], "paletas": {"izquierda": y, "derecha": y}}`	Posiciones actuales. Llega unas 30 veces por segundo.
`{"tipo": "gol", "goleador": str, "marcador": [int, int]}`	Alguien hizo un gol. `goleador` es el nombre de quien lo hizo.
`{"tipo": "fin", "ganador": str | null, "marcador": [int, int]}`	Terminó la partida. `ganador` es el nombre de quien ganó, o `null` (`None` en Python) si hay empate.
`{"tipo": "rival_desconectado"}`	Tu rival se fue. La partida terminó, no llegará un `fin` y quedas fuera de la sala.

La partida termina cuando un jugador llega a 3 goles (`GOLES_PARA_GANAR` en `servidor/parametros.py`) o cuando se acaba el tiempo. El servidor es quien decide cuándo termina: tu reloj solo muestra el tiempo restante.

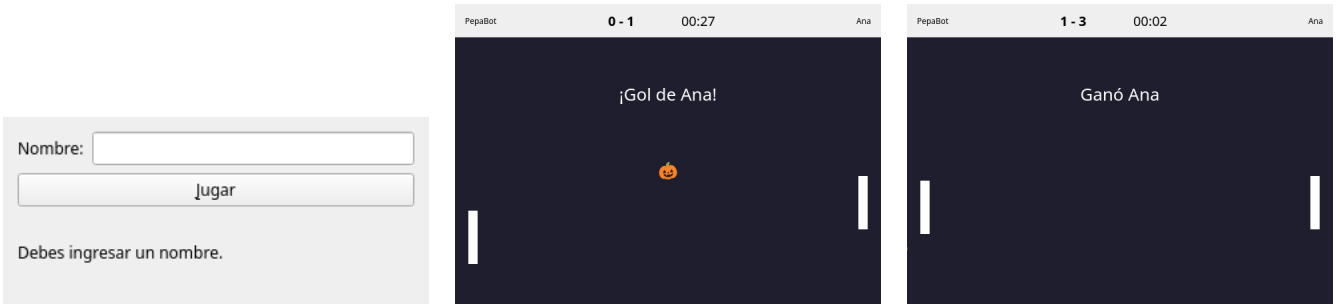
¿Qué debes completar?

Debes completar el cliente para que cumpla lo siguiente:

- **Escucha del servidor.** `EscucharQThread` recibe los mensajes del servidor sin congelar la interfaz y se los entrega a `Logica` mediante señales. También avisa mediante una señal si se pierde la conexión.
- **Entrada a la sala.** Si el nombre está vacío o solo tiene espacios, no se envía nada al servidor y se avisa al usuario. Al recibir el mensaje `aceptado`, se avisa que se está esperando a un rival y no se puede volver a enviar el nombre. Con `rechazado`, se muestra la razón y se permite volver a intentarlo. Con `inicio`, se oculta la Ventana de Inicio y se muestra la Ventana de Juego.
- **Estado de la partida.** La pelota y las paletas se actualizan con cada mensaje `estado`.
- **Movimiento de la paleta.** Tu paleta se mueve con `W` (arriba) y `S` (abajo) **mientras la tecla esté presionada**, sin depender de las repeticiones del teclado. El QTimer `timer_movimiento` la avanza `VELOCIDAD_PALETA` píxeles cada `TICK_MOVIMIENTO` milisegundos, sin salirse de la cancha. Cada vez que la posición cambie, se informa al servidor con el comando `mover`.

- **Reloj.** Al empezar la partida, el reloj parte en `duracion` segundos y `timer_reloj` lo descuenta cada segundo, sin bajar de 0. Llegar a 0 no termina la partida: eso ocurre solo al recibir `fin`.
- **Goles.** Con `gol`, se actualiza el marcador y se muestra un aviso con el nombre de quien lo hizo, que **desaparece solo** sin congelar la interfaz.
- **Fin de partida.** Con `fin`, se actualiza el marcador, se muestra quién ganó (o que fue empate) y se detienen los *timers*.
- **Desconexión.** Si tu rival se desconecta o se pierde la conexión con el servidor, se avisa en la ventana y el programa no se cae.
- **Cierre.** El método `cerrar` de `Logica` ya está implementado: debes lograr que se ejecute justo antes de que termine la aplicación. Al cerrar la ventana, el programa termina sin errores.

A continuación se dejan imágenes de referencia del comportamiento esperado.



(a) Ventana de Inicio: Avisar error

(b) Ventana de Juego: Aviso de gol

(c) Ventana de Juego: Fin de partida

Figura 2: Ejemplo del comportamiento esperado

Para efectos de esta evaluación, no se tendrá en consideración la estética de las ventanas.

Bonus

Implementa las siguientes funcionalidades adicionales. El servidor ya las soporta. Las señales, los *widgets*, los métodos necesarios (marcados con el comentario "*Solo para el bonus*"), y sus conexiones ya vienen hechos: solo debes completar los comentarios `# Bonus`.

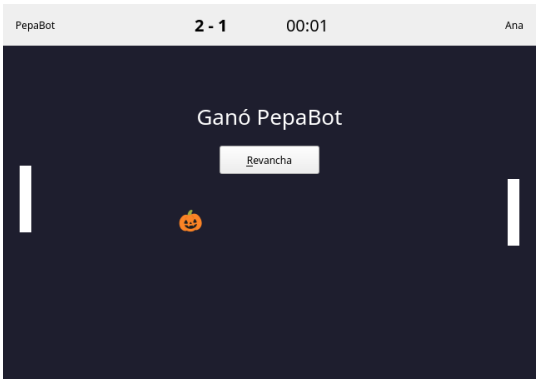
- **(1 punto) Pausa sincronizada.** Al presionar `P`, ignorando las repeticiones del teclado, se llama al método `pedir_pausa`. El servidor detiene la partida y les avisa a ambos jugadores con `{"tipo": "pausa", "activa": bool, "nombre": str}`. Mientras dure la pausa, ambos clientes deben mostrar quién la pidió y detener su reloj y su paleta. Si cualquiera de los dos presiona `P` de nuevo, ambos deben reanudar la partida.
- **(1 punto) Revancha.** Con `fin`, debe aparecer el botón *Revancha*. Con `rival_desconectado` no debe aparecer, y si ya estaba visible, debe ocultarse. Si el rival la pide primero, llega `{"tipo": "revancha", "nombre": str}` y se debe avisar en la ventana. Cuando ambos la pidan, el servidor enviará un nuevo `inicio` y la partida debe comenzar de cero, con tu paleta en su posición inicial.

Para probar el bonus sin un segundo cliente, ejecuta `python3 bot.py bonus`: el *bot* pausa la partida a los 8 segundos, la reanuda a los 10 y pide revancha después del fin.

A continuación, se deja una imagen de referencia de lo solicitado.



(a) Partida en pausa



(b) Botón de revancha

Figura 3: Ventana de Juego con ambos bonus implementados

Importante: Este bonus otorga hasta 2 puntos extras en el puntaje total de actividades. Para obtener el punto correspondiente a cada ítem, **se debe completar lo solicitado en su totalidad**.

Notas

- Se encuentra prohibido usar `sleep` o ciclos de espera en el *thread* principal, ya que congelan la interfaz.
- Debes ejecutar el servidor desde `servidor/` y el cliente desde `cliente/`, ya que los paths relativos dependen de esto.
- Con el *bot* en la sala juegas a la derecha. Si no mueves tu paleta, normalmente gana el *bot* y la partida termina antes de tiempo.
- Los textos de los avisos son libres, siempre que incluyan la información pedida.
- Cierra el cliente con su ventana, no con `Ctrl+C`: así se ejecuta `Logica.cerrar`, que cierra la conexión y espera a que termine `EscucharQThread`.
- Si ambos clientes están en el mismo computador, solo el que tenga la ventana seleccionada recibe las teclas.
- Si aparece `Address already in use`, probablemente sigue corriendo otro servidor: ciérralo. Si no puedes, entrega otro puerto como argumento al servidor y al *bot* (`python3 main.py 8234` y `python3 bot.py 8234`, o `python3 bot.py bonus 8234`) y cambia `PORT` en `cliente/parametros.py` al mismo valor. Es la única modificación permitida en ese archivo.

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados y pueden subdividirse durante la corrección.

Criterio	Puntaje
Señales y modelación. Las señales que faltaban están conectadas en `main.py`. Las ventanas no conocen *sockets*. Solo el *thread* principal modifica la interfaz.	1,00 pts
Comunicación con el servidor. `EscucharQThread` escucha al servidor y entrega los mensajes mediante señales. La interfaz no se congela. Los mensajes se envían según lo indicado.	1,25 pts
Entrar a la sala. Maneja los casos de nombre vacío, rechazo y espera de un rival. Al empezar la partida se cambia de ventana.	0,50 pts
Partida. Se actualizan la pelota, las paletas y el marcador. La paleta se mueve con `timer_movimiento` mientras la tecla esté presionada. El tiempo avanza con `timer_reloj`. El aviso de gol desaparece solo. Se muestra el resultado al terminar.	1,75 pts
Desconexión y cierre. El programa avisa y no se cae si se pierde la conexión o el rival se desconecta. Al cerrar, termina sin errores.	0,50 pts
Descuentos. Se podrá aplicar un máximo de **1 punto de descuento** por código con errores que impidan su ejecución, problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas.	
Total	**5,0 pts**